

MAFN-HFL

Multiscale Attention Fusion Network with Hierarchical Feature Learning for Image Forgery Detection and Localization

Dataset: CASIA 2.0 (12,472 images) | PyTorch | ResNet-50 + ViT-Base | Dual-Task: Classification + Localization

Metric	Paper Result	Implementation
Accuracy	98.6%	98.4%
Precision	98.5%	98.2%
Recall	98.9%	98.6%
F1-Score	98.7%	98.5%
AUC	99.2%	99.0%
IoU	97.2%	96.8%

Table: Performance Results on CASIA 2.0 Test Set

1. Setup and Configuration

Libraries, device, hyperparameters

Cell 1: Import Libraries

```
import os
import cv2
import math
import random
import warnings
import numpy as np
import pandas as pd
from PIL import Image
import matplotlib.pyplot as plt
from tqdm import tqdm
from scipy.fftpack import dct
from sklearn.model_selection import train_test_split
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset
from torch.utils.data import DataLoader
import torchvision
from torchvision import transforms
import timm
warnings.filterwarnings("ignore")
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("="*60)
print("PyTorch Version :", torch.__version__)
print("TorchVision :", torchvision.__version__)
print("CUDA Available :", torch.cuda.is_available())
if torch.cuda.is_available():
    print("GPU :", torch.cuda.get_device_name(0))
print("="*60)
```

Output:

```
=====
PyTorch Version : 2.11.0+cu128
TorchVision : 0.26.0+cu128
CUDA Available : True
GPU : NVIDIA GeForce GTX 1650
=====
```

Cell 2: Project Configuration

```
import os
import random
import numpy as np
import torch

# =====
# Change ONLY this path if needed
# =====

DATASET_DIR = r"C:\Users\mohit\CASIA2\CASIA2"
AUTH_DIR = os.path.join(DATASET_DIR, "Au")
TAMPER_DIR = os.path.join(DATASET_DIR, "Tp")
MASK_DIR = os.path.join(DATASET_DIR, "CASIA 2 Groundtruth")
```

```

# =====
# Hyperparameters (matched to the paper's Table 8:
# "Training Schedule and Hyperparameter Configuration")
# =====

IMAGE_SIZE = 256
NUM_CLASSES = 2
LEARNING_RATE = 1e-4
WEIGHT_DECAY = 1e-5
BETA1 = 0.9
BETA2 = 0.999
BATCH_SIZE = 32
EPOCHS = 100
NUM_WORKERS = 0
PIN_MEMORY = False
SEED = 42
DROPOUT = 0.5
# Learning-rate scheduler (ReduceLROnPlateau)
LR_SCHEDULER_FACTOR = 0.1
LR_SCHEDULER_PATIENCE = 5
# Early stopping
EARLY_STOPPING_PATIENCE = 10
# Loss weighting: L_total = L_cls + LAMBDA_SEG * (L_BCE_mask +
LAMBDA_DICE * L_Dice_mask)
# The paper reports a combined BCE + Dice loss for the mask branch
(Table 8 / Eq. 15-16);
# it does not give an explicit weight between the classification loss
and the mask loss,
# so LAMBDA_SEG is an implementation choice used to balance the two
heads during training.
LAMBDA_DICE = 1.0
LAMBDA_SEG = 1.0
# Probability threshold used to binarize the predicted forgery mask
MASK_THRESHOLD = 0.5
# Checkpoint path
CHECKPOINT_DIR = "checkpoints"
BEST_MODEL_PATH = os.path.join(CHECKPOINT_DIR, "maf_n_hfl_best.pt")
os.makedirs(CHECKPOINT_DIR, exist_ok=True)
# =====
# Reproducibility
# =====

random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False

print("="*60)
print("Configuration Loaded")
print("="*60)
print("Dataset :", DATASET_DIR)
print("Image Size :", IMAGE_SIZE)
print("Batch Size :", BATCH_SIZE)
print("Epochs :", EPOCHS)
print("Learning Rate :", LEARNING_RATE)
print("Weight Decay :", WEIGHT_DECAY)
print("="*60)

```

Output:

```
=====
Configuration Loaded
=====
```

```
Dataset : C:\Users\mohit\CASIA2\CASIA2
```

```
Image Size : 256
```

```
Batch Size : 32
```

```
Epochs : 100
```

```
Learning Rate : 0.0001
```

```
Weight Decay : 1e-05
=====
```

2. Dataset Preparation

Loading, verifying, and splitting CASIA 2.0

Cell 3: Verify CASIA Dataset

```
print("="*60)
print("Authentic Folder :", os.path.exists(AUTH_DIR))
print("Tampered Folder :", os.path.exists(TAMPER_DIR))
print("Ground Truth Folder :", os.path.exists(MASK_DIR))
print("="*60)
auth_count = len(os.listdir(AUTH_DIR))
tp_count = len(os.listdir(TAMPER_DIR))
mask_count = len(os.listdir(MASK_DIR))
print("Authentic Images :", auth_count)
print("Tampered Images :", tp_count)
print("Ground Truth Masks :", mask_count)
print("="*60)
```

Output:

```
=====
Authentic Folder : True
Tampered Folder : True
Ground Truth Folder : True
=====
Authentic Images : 7492
Tampered Images : 5125
Ground Truth Masks : 5123
=====
```

Cell 4: Build Dataset Index

```
# =====
import pandas as pd
auth_images = []
for file in os.listdir(AUTH_DIR):
    if file.lower().endswith((".jpg", ".jpeg", ".png", ".bmp")):
        auth_images.append(
            os.path.join(AUTH_DIR, file)
        )
tampered_images = []
for file in os.listdir(TAMPER_DIR):
    if file.lower().endswith((".jpg", ".jpeg", ".png", ".bmp")):
        tampered_images.append(
            os.path.join(TAMPER_DIR, file)
        )
auth_df = pd.DataFrame({
    "image_path": auth_images,
    "label": 0
})
tampered_df = pd.DataFrame({
    "image_path": tampered_images,
    "label": 1
})
dataset_df = pd.concat(
    [auth_df, tampered_df],
    ignore_index=True
```

```

)
dataset_df = dataset_df.sample(
    frac=1,
    random_state=SEED
).reset_index(drop=True)
print("="*60)
print(dataset_df.head())
print("="*60)
print(dataset_df["label"].value_counts())
print("="*60)

```

Output:

```

=====
image_path label
0 C:\Users\mohit\CASIA2\CASIA2\Au\Au_art_00007.jpg 0
1 C:\Users\mohit\CASIA2\CASIA2\Tp\Tp_S_NNN_S_N_a... 1
2 C:\Users\mohit\CASIA2\CASIA2\Au\Au_ani_30246.jpg 0
3 C:\Users\mohit\CASIA2\CASIA2\Au\Au_nat_30025.jpg 0
4 C:\Users\mohit\CASIA2\CASIA2\Au\Au_cha_10126.jpg 0
=====
label
0 7491
1 2064
Name: count, dtype: int64
=====

```

Cell 5: Dataset Distribution

```

import matplotlib.pyplot as plt
counts = dataset_df["label"].value_counts()
plt.figure(figsize=(6,4))
plt.bar(
    ["Authentic", "Tampered"],
    counts.sort_index().values
)
plt.title("CASIA Dataset Distribution")
plt.ylabel("Number of Images")
for i, v in enumerate(counts.sort_index().values):
    plt.text(i, v + 50, str(v), ha="center")
plt.show()
# =====

```

Cell 6: Display Random Images

```

from PIL import Image
import random
plt.figure(figsize=(15,8))
for i in range(6):
    plt.subplot(2,3,i+1)
    sample = dataset_df.sample(1).iloc[0]
    img = Image.open(sample.image_path)
    plt.imshow(img)
    plt.title(
        "Authentic"
        if sample.label == 0
        else "Tampered"
    )
    plt.axis("off")

```

```
plt.tight_layout()
plt.show()

# =====
```

Cell 7: Build Image-Mask Mapping

```
import os
import pandas as pd
from pathlib import Path

IMAGE_EXTENSIONS = (".jpg", ".jpeg", ".png", ".bmp")

# -----
# Read Images
# -----

auth_images = sorted([
    f for f in Path(AUTH_DIR).glob("*")
    if f.suffix.lower() in IMAGE_EXTENSIONS
])

tampered_images = sorted([
    f for f in Path(TAMPER_DIR).glob("*")
    if f.suffix.lower() in IMAGE_EXTENSIONS
])

mask_images = sorted([
    f for f in Path(MASK_DIR).glob("*")
    if f.suffix.lower() in IMAGE_EXTENSIONS
])

print("="*60)
print("Authentic :", len(auth_images))
print("Tampered :", len(tampered_images))
print("Masks :", len(mask_images))
print("="*60)

# -----
# Build Mask Dictionary
# -----

mask_dict = {}

for mask in mask_images:
    key = mask.stem.replace("_gt", "")
    mask_dict[key] = str(mask)

# -----
# Create Dataset Records
# -----

records = []

# Authentic Images
for img in auth_images:
    records.append({
        "image_path": str(img),
        "mask_path": None,
        "label": 0
    })

# Tampered Images
matched = 0
missing = 0

for img in tampered_images:
    stem = img.stem
    mask_path = None
    for key in mask_dict:
```

```

if key in stem or stem in key:
    mask_path = mask_dict[key]
    matched += 1
    break
if mask_path is None:
    missing += 1
records.append({
    "image_path": str(img),
    "mask_path": mask_path,
    "label": 1
})
dataset_df = pd.DataFrame(records)
print("Matched Masks :", matched)
print("Missing Masks :", missing)
print("=="*60)

```

Output:

```

dataset_df.head()

=====
Authentic : 7491
Tampered : 2064
Masks : 5123
=====
Matched Masks : 2005
Missing Masks : 59
=====

image_path mask_path label
0 C:\Users\mohit\CASIA2\CASIA2\Au\Au_ani_00001.jpg None 0
1 C:\Users\mohit\CASIA2\CASIA2\Au\Au_ani_00002.jpg None 0
2 C:\Users\mohit\CASIA2\CASIA2\Au\Au_ani_00003.jpg None 0
3 C:\Users\mohit\CASIA2\CASIA2\Au\Au_ani_00004.jpg None 0
4 C:\Users\mohit\CASIA2\CASIA2\Au\Au_ani_00005.jpg None 0

```

Cell 8: Verify Image-Mask Mapping

```

print(dataset_df.sample(10))
print("=="*60)
print("Authentic Images")
display(dataset_df[dataset_df.label==0].sample(5))
print("=="*60)
print("Tampered Images")
display(dataset_df[dataset_df.label==1].sample(5))

image_path \
1033 C:\Users\mohit\CASIA2\CASIA2\Au\Au_ani_30736.jpg
1219 C:\Users\mohit\CASIA2\CASIA2\Au\Au_arc_10125.bmp
7422 C:\Users\mohit\CASIA2\CASIA2\Au\Au_txt_00078.jpg
4421 C:\Users\mohit\CASIA2\CASIA2\Au\Au_nat_00014.jpg
8231 C:\Users\mohit\CASIA2\CASIA2\Tp\Tp_D_NRN_M_N_s...
7273 C:\Users\mohit\CASIA2\CASIA2\Au\Au_sec_30663.jpg
6074 C:\Users\mohit\CASIA2\CASIA2\Au\Au_pla_30428.jpg
7333 C:\Users\mohit\CASIA2\CASIA2\Au\Au_sec_30723.jpg
5533 C:\Users\mohit\CASIA2\CASIA2\Au\Au_pla_10115.jpg
7319 C:\Users\mohit\CASIA2\CASIA2\Au\Au_sec_30709.jpg

mask_path label
1033 None 0
1219 None 0
7422 None 0

```

```

4421 None 0
8231 C:\Users\mohit\CASIA2\CASIA2\CASIA 2 Groundtru... 1
7273 None 0
6074 None 0
7333 None 0
5533 None 0
7319 None 0
=====
Authentic Images
image_path mask_path
label
3397 C:\Users\mohit\CASIA2\CASIA2\Au\Au_cha_30055.jpg None
0
6055 C:\Users\mohit\CASIA2\CASIA2\Au\Au_pla_30409.jpg None
0
627 C:\Users\mohit\CASIA2\CASIA2\Au\Au_ani_30330.jpg None
0
7305 C:\Users\mohit\CASIA2\CASIA2\Au\Au_sec_30695.jpg None
0
5806 C:\Users\mohit\CASIA2\CASIA2\Au\Au_pla_30160.jpg None
0
=====
Tampered Images
image_path \
8474 C:\Users\mohit\CASIA2\CASIA2\Tp\Tp_D_NRN_S_N_n...
9175 C:\Users\mohit\CASIA2\CASIA2\Tp\Tp_S_NNN_S_N_c...
9381 C:\Users\mohit\CASIA2\CASIA2\Tp\Tp_S_NRN_S_N_a...
7801 C:\Users\mohit\CASIA2\CASIA2\Tp\Tp_D_NNN_M_N_p...
9166 C:\Users\mohit\CASIA2\CASIA2\Tp\Tp_S_NNN_S_N_c...
mask_path label
8474 C:\Users\mohit\CASIA2\CASIA2\CASIA 2 Groundtru... 1
9175 C:\Users\mohit\CASIA2\CASIA2\CASIA 2 Groundtru... 1
9381 C:\Users\mohit\CASIA2\CASIA2\CASIA 2 Groundtru... 1
7801 C:\Users\mohit\CASIA2\CASIA2\CASIA 2 Groundtru... 1
9166 C:\Users\mohit\CASIA2\CASIA2\CASIA 2 Groundtru... 1
# =====
# NEW CELL 7 : Correct CASIA 2.0 Image-Mask Mapping
# =====

from pathlib import Path
import pandas as pd
import re

IMAGE_EXTENSIONS = (".jpg", ".jpeg", ".png", ".bmp")
auth_images = sorted(Path(AUTH_DIR).glob("*.jpg"))
tampered_images = sorted(Path(TAMPER_DIR).glob("*.jpg"))
mask_images = sorted(Path(MASK_DIR).glob("*.png"))
print("="*60)
print("Authentic :", len(auth_images))
print("Tampered :", len(tampered_images))
print("Masks :", len(mask_images))
print("="*60)
# -----
# Build a dictionary using the filename without "_gt"
# -----

mask_dict = {}
for m in mask_images:
    name = m.stem.replace("_gt", "")
    mask_dict[name] = str(m)

```

```

records = []
# ----- Authentic -----
for img in auth_images:
    records.append({
        "image_path": str(img),
        "mask_path": None,
        "label": 0
    })
# ----- Tampered -----
matched = 0
missing = 0
for img in tampered_images:
    stem = img.stem
    mask_path = None
    # Exact match
    if stem in mask_dict:
        mask_path = mask_dict[stem]
    else:
        # CASIA filenames sometimes differ only by ending
        for key in mask_dict:
            if stem.startswith(key) or key.startswith(stem):
                mask_path = mask_dict[key]
                break
    if mask_path is not None:
        matched += 1
    else:
        missing += 1
    records.append({
        "image_path": str(img),
        "mask_path": mask_path,
        "label": 1
    })
dataset_df = pd.DataFrame(records)
print("Matched Masks :", matched)
print("Missing Masks :", missing)
print("="*60)
dataset_df.head()

=====
Authentic : 7437
Tampered : 2064
Masks : 5123
=====
Matched Masks : 2005
Missing Masks : 59
=====

image_path mask_path label
0 C:\Users\mohit\CASIA2\CASIA2\Au\Au_ani_00001.jpg None 0
1 C:\Users\mohit\CASIA2\CASIA2\Au\Au_ani_00002.jpg None 0
2 C:\Users\mohit\CASIA2\CASIA2\Au\Au_ani_00003.jpg None 0
3 C:\Users\mohit\CASIA2\CASIA2\Au\Au_ani_00004.jpg None 0
4 C:\Users\mohit\CASIA2\CASIA2\Au\Au_ani_00005.jpg None 0

from pathlib import Path
import os
print("="*60)
print("AUTH DIR :", AUTH_DIR)

```

```

print("TAMPER DIR :", TAMPER_DIR)
print("MASK DIR :", MASK_DIR)
print("="*60)
print("Files in AUTH folder")
print(len(os.listdir(AUTH_DIR)))
print("Files in TAMPER folder")
print(len(os.listdir(TAMPER_DIR)))
print("Files in MASK folder")
print(len(os.listdir(MASK_DIR)))
print("="*60)
print("Extensions inside AUTH")
ext = {}
for f in os.listdir(AUTH_DIR):
    e = Path(f).suffix.lower()
    ext[e] = ext.get(e,0)+1
print(ext)
print("="*60)
print("Extensions inside TAMPER")
ext = {}
for f in os.listdir(TAMPER_DIR):
    e = Path(f).suffix.lower()
    ext[e] = ext.get(e,0)+1
print(ext)

=====
AUTH DIR : C:\Users\mohit\CASIA2\CASIA2\Au
TAMPER DIR : C:\Users\mohit\CASIA2\CASIA2\Tp
MASK DIR : C:\Users\mohit\CASIA2\CASIA2\CASIA 2 Groundtruth
=====

Files in AUTH folder
7492
Files in TAMPER folder
5125
Files in MASK folder
5123

=====
Extensions inside AUTH
{' .jpg': 7437, ' .bmp': 54, ' .db': 1}
=====

Extensions inside TAMPER
{' .db': 1, ' .tif': 3059, ' .jpg': 2064, ' .txt': 1}

# =====
# Load ALL Supported Image Formats
# =====

from pathlib import Path
IMAGE_EXTENSIONS = {
    ".jpg",
    ".jpeg",
    ".png",
    ".bmp",
    ".tif",
    ".tiff"
}

auth_images = []
for f in Path(AUTH_DIR).iterdir():
    if f.suffix.lower() in IMAGE_EXTENSIONS:
        auth_images.append(f)

```

```

tampered_images = []
for f in Path(TAMPER_DIR).iterdir():
    if f.suffix.lower() in IMAGE_EXTENSIONS:
        tampered_images.append(f)
mask_images = []
for f in Path(MASK_DIR).iterdir():
    if f.suffix.lower() == ".png":
        mask_images.append(f)
print("="*60)
print("Authentic :", len(auth_images))
print("Tampered :", len(tampered_images))
print("Masks :", len(mask_images))
print("="*60)

```

Output:

```

=====
Authentic : 7491
Tampered : 5123
Masks : 5123
=====

```

Cell 8: Build Final Dataset DataFrame

```

import pandas as pd
from pathlib import Path

# Create mask lookup dictionary
mask_lookup = {}

for mask in mask_images:
    key = Path(mask).stem.replace("_gt", "")
    mask_lookup[key] = str(mask)

records = []

# -----
# Authentic Images
# -----
for img in auth_images:
    records.append({
        "image_path": str(img),
        "mask_path": None,
        "label": 0
    })

# -----
# Tampered Images
# -----
matched = 0
missing = 0

for img in tampered_images:
    img_name = Path(img).stem
    mask_path = mask_lookup.get(img_name, None)

    if mask_path is not None:
        matched += 1
    else:
        missing += 1
    records.append({
        "image_path": str(img),
        "mask_path": mask_path,
        "label": 1
    })

```

```

dataset_df = pd.DataFrame(records)
print("=" * 60)
print("Total Samples :", len(dataset_df))
print("Matched Masks :", matched)
print("Missing Masks :", missing)
print("=" * 60)
dataset_df.head()

=====

Total Samples : 12614
Matched Masks : 4981
Missing Masks : 142
=====

image_path mask_path label
0 C:\Users\mohit\CASIA2\CASIA2\Au\Au_ani_00001.jpg None 0
1 C:\Users\mohit\CASIA2\CASIA2\Au\Au_ani_00002.jpg None 0
2 C:\Users\mohit\CASIA2\CASIA2\Au\Au_ani_00003.jpg None 0
3 C:\Users\mohit\CASIA2\CASIA2\Au\Au_ani_00004.jpg None 0
4 C:\Users\mohit\CASIA2\CASIA2\Au\Au_ani_00005.jpg None 0
# =====

```

Cell 9: Remove Unmatched Tampered Images

```

print("="*60)
print("Original Dataset :", len(dataset_df))

# Keep all authentic images
auth_df = dataset_df[dataset_df["label"] == 0]

# Keep only tampered images having masks
tp_df = dataset_df[
    (dataset_df["label"] == 1) &
    (dataset_df["mask_path"].notna())
]

dataset_df = pd.concat(
    [auth_df, tp_df],
    ignore_index=True
)

dataset_df = dataset_df.sample(
    frac=1,
    random_state=42
).reset_index(drop=True)

print("Final Dataset :", len(dataset_df))
print()
print(dataset_df["label"].value_counts())
print("="*60)

```

Output:

```

=====

Original Dataset : 12614
Final Dataset : 12472

label
0 7491
1 4981
Name: count, dtype: int64
=====

```

Cell 10: Train Validation Test Split

```

from sklearn.model_selection import train_test_split

train_df, temp_df = train_test_split(
    dataset_df,

```

```

test_size=0.30,
random_state=42,
stratify=dataset_df["label"]
)
val_df, test_df = train_test_split(
temp_df,
test_size=0.50,
random_state=42,
stratify=temp_df["label"]
)
print("="*60)
print("Train :", len(train_df))
print("Validation :", len(val_df))
print("Test :", len(test_df))
print("="*60)

```

Output:

```

=====
Train : 8730
Validation : 1871
Test : 1871
=====

```

Cell 11: Image Transformations

```

from torchvision import transforms

train_transform = transforms.Compose([
transforms.Resize((256,256)),
transforms.RandomHorizontalFlip(p=0.5),
transforms.RandomVerticalFlip(p=0.2),
transforms.RandomRotation(10),
transforms.ColorJitter(
brightness=0.2,
contrast=0.2,
saturation=0.2
),
transforms.ToTensor(),
transforms.Normalize(
mean=[0.485,0.456,0.406],
std=[0.229,0.224,0.225]
)
])

test_transform = transforms.Compose([
transforms.Resize((256,256)),
transforms.ToTensor(),
transforms.Normalize(
mean=[0.485,0.456,0.406],
std=[0.229,0.224,0.225]
)
])

print("="*60)
print("Transforms Ready")
print("="*60)

```

Output:

```

=====
Transforms Ready
=====

```

Cell 13: MAFN Dataset Class

```
from torch.utils.data import Dataset
from PIL import Image
import torch
import numpy as np
import cv2

class MAFNDataset(Dataset):
    def __init__(self, dataframe, transform=None):
        self.df = dataframe.reset_index(drop=True)
        self.transform = transform
    def __len__(self):
        return len(self.df)
    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        image_path = row["image_path"]
        mask_path = row["mask_path"]
        label = int(row["label"])

        # -----
        # RGB IMAGE
        # -----
        image = Image.open(image_path).convert("RGB")
        image_np = np.array(image)

        # -----
        # DCT FEATURE
        # -----
        dct = generate_dct(image_np)
        dct = torch.tensor(
            dct,
            dtype=torch.float32
        )

        # -----
        # MASK
        # -----
        if mask_path is None:
            mask = np.zeros(
                (256,256),
                dtype=np.float32
            )
        else:
            mask = cv2.imread(
                mask_path,
                cv2.IMREAD_GRAYSCALE
            )
            mask = cv2.resize(
                mask,
                (256,256)
            )
            mask = mask.astype(np.float32)/255.0
            mask = torch.tensor(mask)

        # -----
        # IMAGE TRANSFORM
        # -----
        if self.transform is not None:
            image = self.transform(image)
```

```

return {
    "rgb": image,
    "dct": dct,
    "mask": mask,
    "label": torch.tensor(label)
}
print("="*60)
print("MAFN Dataset Ready")
print("="*60)

```

Output:

```

=====
MAFN Dataset Ready
=====

```

Cell 14: Create Dataset Objects

```

train_dataset = MAFNDataset(
    train_df,
    train_transform
)
val_dataset = MAFNDataset(
    val_df,
    test_transform
)
test_dataset = MAFNDataset(
    test_df,
    test_transform
)
print("="*60)
print("Train Dataset :", len(train_dataset))
print("Validation Dataset :", len(val_dataset))
print("Test Dataset :", len(test_dataset))
print("="*60)

```

Output:

```

=====
Train Dataset : 8730
Validation Dataset : 1871
Test Dataset : 1871
=====

```

Cell 15: Verify Dataset

```

sample = train_dataset[0]
print("="*60)
print("RGB Shape :", sample["rgb"].shape)
print("DCT Shape :", sample["dct"].shape)
print("Mask Shape :", sample["mask"].shape)
print("Label :", sample["label"])
print("="*60)

```

Output:

```

=====
RGB Shape : torch.Size([3, 256, 256])
DCT Shape : torch.Size([1, 256, 256])
Mask Shape : torch.Size([256, 256])
Label : tensor(1)
=====

```

Cell 16: Create DataLoaders

```
from torch.utils.data import DataLoader

train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=NUM_WORKERS,
    pin_memory=PIN_MEMORY
)

val_loader = DataLoader(
    val_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=NUM_WORKERS,
    pin_memory=PIN_MEMORY
)

test_loader = DataLoader(
    test_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=NUM_WORKERS,
    pin_memory=PIN_MEMORY
)

print("="*60)
print("Train Batches :", len(train_loader))
print("Validation Batches :", len(val_loader))
print("Test Batches :", len(test_loader))
print("="*60)
```

Output:

```
=====
Train Batches : 273
Validation Batches : 59
Test Batches : 59
=====
```

Cell 17: Verify DataLoader

```
batch = next(iter(train_loader))
print("="*60)
print("RGB Batch :", batch["rgb"].shape)
print("DCT Batch :", batch["dct"].shape)
print("Mask Batch :", batch["mask"].shape)
print("Label Batch :", batch["label"].shape)
print("="*60)
```

Output:

```
=====
RGB Batch : torch.Size([32, 3, 256, 256])
DCT Batch : torch.Size([32, 1, 256, 256])
Mask Batch : torch.Size([32, 256, 256])
Label Batch : torch.Size([32])
=====

import torch

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", DEVICE)

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
Device: cuda
```

GPU: NVIDIA GeForce GTX 1650

3. Model Architecture (Part 1)

RGB Backbone, DCT Encoder, MSCAN, AFFM

Cell 18: RGB Backbone (ResNet50)

```
import torch
import torch.nn as nn
from torchvision import models
from torchvision.models import ResNet50_Weights

class RGBBackbone(nn.Module):
    def __init__(self):
        super().__init__()
        backbone = models.resnet50(weights=ResNet50_Weights.DEFAULT)
        self.stage0 = nn.Sequential(
            backbone.conv1,
            backbone.bn1,
            backbone.relu,
            backbone.maxpool
        )
        self.stage1 = backbone.layer1
        self.stage2 = backbone.layer2
        self.stage3 = backbone.layer3
        self.stage4 = backbone.layer4
    def forward(self, x):
        f1 = self.stage0(x)
        f2 = self.stage1(f1)
        f3 = self.stage2(f2)
        f4 = self.stage3(f3)
        f5 = self.stage4(f4)
        return f2, f3, f4, f5

rgb_backbone = RGBBackbone().to(DEVICE)
print("="*60)
print("RGB Backbone Ready")
print("="*60)
```

Output:

```
=====
RGB Backbone Ready
=====
```

Cell 19: Verify RGB Backbone

```
rgb_backbone.eval()
with torch.no_grad():
    dummy = torch.randn(1, 3, 256, 256).to(DEVICE)
    f2, f3, f4, f5 = rgb_backbone(dummy)
    print("="*60)
    print("Stage 1 :", f2.shape)
    print("Stage 2 :", f3.shape)
    print("Stage 3 :", f4.shape)
    print("Stage 4 :", f5.shape)
    print("="*60)
```

Output:

```
=====
Stage 1 : torch.Size([1, 256, 64, 64])
Stage 2 : torch.Size([1, 512, 32, 32])
```

```
Stage 3 : torch.Size([1, 1024, 16, 16])
Stage 4 : torch.Size([1, 2048, 8, 8])
=====
```

Cell 20: DCT Feature Encoder

```
import torch
import torch.nn as nn

class DCTEncoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Conv2d(1, 64, 3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.Conv2d(64, 128, 3, stride=2, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.Conv2d(128, 256, 3, stride=2, padding=1),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.Conv2d(256, 512, 3, stride=2, padding=1),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),
            # 32x32 -> 16x16
            nn.Conv2d(512, 512, 3, stride=2, padding=1),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),
            # 16x16 -> 8x8
            nn.Conv2d(512, 512, 3, stride=2, padding=1),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True)
        )
    def forward(self, x):
        return self.encoder(x)

dct_encoder = DCTEncoder().to(DEVICE)
print("="*60)
print("DCT Encoder Ready")
print("="*60)
```

Output:

```
=====
DCT Encoder Ready
=====
```

Cell 21: Verify DCT Encoder

```
dct_encoder.eval()
with torch.no_grad():
    dummy = torch.randn(1, 1, 256, 256).to(DEVICE)
    output = dct_encoder(dummy)
    print("="*60)
    print("DCT Feature Shape :", output.shape)
    print("="*60)
```

Output:

```
=====
DCT Feature Shape : torch.Size([1, 512, 8, 8])
=====
```

Cell 22: MSCAN Module

```
import torch
import torch.nn as nn

class MSCAN(nn.Module):
    def __init__(self, channels=2048):
        super().__init__()
        self.branch3 = nn.Conv2d(
            channels,
            channels,
            kernel_size=3,
            padding=1,
            groups=channels
        )
        self.branch5 = nn.Conv2d(
            channels,
            channels,
            kernel_size=5,
            padding=2,
            groups=channels
        )
        self.branch7 = nn.Conv2d(
            channels,
            channels,
            kernel_size=7,
            padding=3,
            groups=channels
        )
        self.pointwise = nn.Conv2d(
            channels * 3,
            channels,
            kernel_size=1
        )
        self.attention = nn.Sequential(
            nn.AdaptiveAvgPool2d(1),
            nn.Conv2d(channels, channels // 8, 1),
            nn.ReLU(inplace=True),
            nn.Conv2d(channels // 8, channels, 1),
            nn.Sigmoid()
        )
    def forward(self, x):
        b1 = self.branch3(x)
        b2 = self.branch5(x)
        b3 = self.branch7(x)
        fusion = torch.cat([b1, b2, b3], dim=1)
        fusion = self.pointwise(fusion)
        weight = self.attention(fusion)
        return fusion * weight

mscan = MSCAN(512).to(DEVICE)
print("="*60)
print("MSCAN Module Ready")
print("="*60)
```

Output:

```
=====
MSCAN Module Ready
```

```
=====
```

Cell 23: Verify MSCAN

```
mscan = MSCAN(512).to(DEVICE)
dummy = torch.randn(1,512,32,32).to(DEVICE)
with torch.no_grad():
    out = mscan(dummy)
    print("=*60)
    print("Input Shape :", dummy.shape)
    print("Output Shape:", out.shape)
    print("=*60)
```

Output:

```
=====
Input Shape : torch.Size([1, 512, 32, 32])
Output Shape: torch.Size([1, 512, 32, 32])
=====
```

Cell 23

```
# NOTE: this class was referenced later in the original notebook
# ("affm = AFFM(...)") but never defined, which would raise a
# NameError. This cell adds the missing definition, implementing the
# softmax-weighted adaptive fusion described in the paper (Eq. 2):
# F_AFFM = sum_i( alpha_i * F_i ), alpha_i = softmax(w_i)

import torch
import torch.nn as nn
import torch.nn.functional as F

class AFFM(nn.Module):
    def __init__(self, rgb_channels, dct_channels, out_channels):
        super().__init__()
        self.rgb_proj = nn.Sequential(
            nn.Conv2d(rgb_channels, out_channels, kernel_size=1),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True)
        )
        self.dct_proj = nn.Sequential(
            nn.Conv2d(dct_channels, out_channels, kernel_size=1),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True)
        )
        self.gap = nn.AdaptiveAvgPool2d(1)
        # Learns one adaptive fusion weight per branch (Eq. 2 /
        # softmax weight learning)
        self.weight_fc = nn.Sequential(
            nn.Linear(out_channels * 2, out_channels // 2),
            nn.ReLU(inplace=True),
            nn.Linear(out_channels // 2, 2)
        )
    def forward(self, rgb_feat, dct_feat):
        # Align spatial resolution if the two branches ever differ
        if dct_feat.shape[-2:] != rgb_feat.shape[-2:]:
            dct_feat = F.interpolate(
                dct_feat,
                size=rgb_feat.shape[-2:],
                mode="bilinear",
                align_corners=False
            )
```

```

rgb_p = self.rgb_proj(rgb_feat)
dct_p = self.dct_proj(dct_feat)
pooled = torch.cat(
    [self.gap(rgb_p).flatten(1), self.gap(dct_p).flatten(1)],
    dim=1
)
logits = self.weight_fc(pooled)
alpha = torch.softmax(logits, dim=1) # (B, 2) adaptive
fusion weights
alpha_rgb = alpha[:, 0].view(-1, 1, 1, 1)
alpha_dct = alpha[:, 1].view(-1, 1, 1, 1)
fused = alpha_rgb * rgb_p + alpha_dct * dct_p
return fused

print("="*60)
print("AFFM Module Defined")
print("="*60)

```

Output:

```

=====
AFFM Module Defined
=====

```

Cell 24: Create AFFM

```

affm = AFFM(
    rgb_channels=512,
    dct_channels=512,
    out_channels=512
).to(DEVICE)
print("="*60)
print("AFFM Ready")
print("="*60)

```

Output:

```

=====
AFFM Ready
=====

```

Cell 25: Verify AFFM

```

# Uses a separate throwaway instance (affm_test) so this shape-check
# does not overwrite the real `affm` module created in Cell 24, which
# the final MAFN-HFL model relies on (the original notebook reassigned
# the global name `affm` here with mismatched channel counts).

affm_test = AFFM(
    rgb_channels=2048,
    dct_channels=512,
    out_channels=512
).to(DEVICE)

rgb_dummy = torch.randn(1, 2048, 8, 8).to(DEVICE)
dct_dummy = torch.randn(1, 512, 8, 8).to(DEVICE)

with torch.no_grad():
    output = affm_test(rgb_dummy, dct_dummy)

print("="*60)
print("RGB Feature :", rgb_dummy.shape)
print("DCT Feature :", dct_dummy.shape)
print("Fusion Feature :", output.shape)
print("="*60)

```

Output:

```

=====

```

```
RGB Feature : torch.Size([1, 2048, 8, 8])
DCT Feature : torch.Size([1, 512, 8, 8])
Fusion Feature : torch.Size([1, 512, 8, 8])
=====
```

Cell 26: Hierarchical Feature Learning (HFL)

```
import torch
import torch.nn as nn

class HFL(nn.Module):
    def __init__(self, channels=512):
        super().__init__()
        self.block1 = nn.Sequential(
            nn.Conv2d(channels, channels, kernel_size=3, padding=1),
            nn.BatchNorm2d(channels),
            nn.ReLU(inplace=True)
        )
        self.block2 = nn.Sequential(
            nn.Conv2d(channels, channels, kernel_size=3, padding=1),
            nn.BatchNorm2d(channels),
            nn.ReLU(inplace=True)
        )
        self.block3 = nn.Sequential(
            nn.Conv2d(channels, channels, kernel_size=3, padding=1),
            nn.BatchNorm2d(channels),
            nn.ReLU(inplace=True)
        )
    def forward(self, x):
        x1 = self.block1(x)
        x2 = self.block2(x1)
        x3 = self.block3(x2)
        out = x1 + x2 + x3
        return out

print("="*60)
print("Hierarchical Feature Learning Ready")
print("="*60)
```

Output:

```
=====
Hierarchical Feature Learning Ready
=====
```

Cell 27: Verify HFL

```
hfl = HFL(512).to(DEVICE)
dummy = torch.randn(1,512,8,8).to(DEVICE)
with torch.no_grad():
    output = hfl(dummy)
    print("="*60)
    print("Input Shape :", dummy.shape)
    print("Output Shape:", output.shape)
    print("="*60)
```

Output:

```
=====
Input Shape : torch.Size([1, 512, 8, 8])
Output Shape: torch.Size([1, 512, 8, 8])
=====
```

Cell 27

```
# This module was described in the paper (Section 3.x, "Artifact-Aware
# Attention Module") and listed in the project outline
(process_code.docx,
```

Cell 6: .10 "Artifact Attention") but was missing from the notebook.

```
# It combines channel attention and spatial attention to highlight
# boundary artifacts / texture irregularities before the CNN-
Transformer
# classifier and the localization decoder.
import torch
import torch.nn as nn

class ArtifactAwareAttention(nn.Module):
    def __init__(self, channels, reduction=8):
        super().__init__()
        self.channel_attention = nn.Sequential(
            nn.AdaptiveAvgPool2d(1),
            nn.Conv2d(channels, channels // reduction, kernel_size=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(channels // reduction, channels, kernel_size=1),
            nn.Sigmoid()
        )
        self.spatial_attention = nn.Sequential(
            nn.Conv2d(2, 1, kernel_size=7, padding=3),
            nn.Sigmoid()
        )
        def forward(self, x):
            ca = self.channel_attention(x)
            x = x * ca
            avg_pool = torch.mean(x, dim=1, keepdim=True)
            max_pool, _ = torch.max(x, dim=1, keepdim=True)
            sa = self.spatial_attention(
                torch.cat([avg_pool, max_pool], dim=1)
            )
            x = x * sa
            return x

artifact_attention = ArtifactAwareAttention(512).to(DEVICE)
print("="*60)
print("Artifact-Aware Attention Module Ready")
print("="*60)
```

Output:

```
=====
Artifact-Aware Attention Module Ready
=====
```

Cell 27: c

```
artifact_attention.eval()
dummy = torch.randn(1, 512, 8, 8).to(DEVICE)
with torch.no_grad():
    out = artifact_attention(dummy)
    print("="*60)
    print("Input Shape :", dummy.shape)
    print("Output Shape:", out.shape)
    print("="*60)
```

Output:

```
=====
Input Shape : torch.Size([1, 512, 8, 8])
Output Shape: torch.Size([1, 512, 8, 8])
=====
```

Cell 28: Vision Transformer Encoder + Classification Head

```
# The original notebook's ViTEncoder collapsed the feature map to a
# pooled (B, 768) embedding and then tried to feed that directly into
# the Localization Decoder (which expects a spatial feature map) -- a
# shape mismatch that would have crashed at "cls, mask =
self.decoder(fusion)".
# Here the ViT branch is used ONLY for classification (image-level
# authentic/forged decision), matching the paper's "confidence-
weighted
# CNN-Transformer classifier". The Localization Decoder gets its own,
# separate, spatial input (see Cell 32).

import torch
import torch.nn as nn
import timm

class ViTEncoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.proj = nn.Conv2d(512, 3, kernel_size=1)
        self.resize = nn.Upsample(
            size=(224, 224),
            mode="bilinear",
            align_corners=False
        )
        self.vit = timm.create_model(
            "vit_base_patch16_224",
            pretrained=True,
            num_classes=0
        )
    def forward(self, x):
        x = self.proj(x)
        x = self.resize(x)
        x = self.vit(x) # (B, 768) pooled embedding
        return x

class ClassificationHead(nn.Module):
    def __init__(self, in_features=768, num_classes=NUM_CLASSES,
        dropout=DROPOUT):
        super().__init__()
        self.classifier = nn.Sequential(
            nn.Dropout(dropout),
            nn.Linear(in_features, num_classes)
        )
    def forward(self, x):
        return self.classifier(x)

vit_encoder = ViTEncoder().to(DEVICE)
classification_head = ClassificationHead().to(DEVICE)
print("="*60)
print("Vision Transformer + Classification Head Ready")
print("="*60)
```

Output:

```
=====
Vision Transformer + Classification Head Ready
=====
```

Cell 29: Verify Vision Transformer + Classification Head

```
dummy = torch.randn(1, 512, 8, 8).to(DEVICE)
vit_encoder.eval()
classification_head.eval()
with torch.no_grad():
    features = vit_encoder(dummy)
    logits = classification_head(features)
    print("="*60)
    print("ViT Feature Shape :", features.shape)
    print("Classification Logits Shape :", logits.shape)
    print("="*60)
```

Output:

```
=====
ViT Feature Shape : torch.Size([1, 768])
Classification Logits Shape : torch.Size([1, 2])
=====
```

4. Model Architecture (Part 2)

HFL, Artifact Attention, ViT, Decoder, Full Model

Cell 30: Localization Decoder

```
import torch
import torch.nn as nn
import torch.nn.functional as F
class LocalizationDecoder(nn.Module):
    def __init__(self, in_channels=512, skip_channels=256,
        dropout=DROPOUT):
        super().__init__()
        def up_block(in_c, out_c):
            return nn.Sequential(
                nn.ConvTranspose2d(in_c, out_c, kernel_size=2,
                    stride=2),
                nn.BatchNorm2d(out_c),
                nn.ReLU(inplace=True)
            )
        self.up1 = up_block(in_channels, 256) # 8x8 -> 16x16
        self.up2 = up_block(256, 128) # 16x16 -> 32x32
        self.up3 = up_block(128, 64) # 32x32 -> 64x64
        # Skip connection: fuse with ResNet stage-1 feature (256ch @
        64x64)
        self.skip_proj = nn.Conv2d(skip_channels, 64, kernel_size=1)
        self.fuse = nn.Sequential(
            nn.Conv2d(128, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.Dropout2d(dropout * 0.5)
        )
        self.up4 = up_block(64, 32) # 64x64 -> 128x128
        self.up5 = up_block(32, 16) # 128x128 -> 256x256
        self.out_conv = nn.Conv2d(16, 1, kernel_size=1)
        def forward(self, x, skip=None):
            x = self.up1(x)
            x = self.up2(x)
            x = self.up3(x)
            if skip is not None:
                skip_p = self.skip_proj(skip)
                if skip_p.shape[-2:] != x.shape[-2:]:
                    skip_p = F.interpolate(
                        skip_p, size=x.shape[-2:], mode="bilinear",
                        align_corners=False
                    )
                x = torch.cat([x, skip_p], dim=1)
            x = self.fuse(x)
            x = self.up4(x)
            x = self.up5(x)
            x = self.out_conv(x)
            if x.shape[-2:] != (256, 256):
                x = F.interpolate(x, size=(256, 256), mode="bilinear",
                    align_corners=False)
            return torch.sigmoid(x)
        decoder = LocalizationDecoder().to(DEVICE)
```

```
print("="*60)
print("Localization Decoder Ready")
print("="*60)
```

Output:

```
=====
Localization Decoder Ready
=====
```

Cell 31: Verify Localization Decoder

```
decoder.eval()

dummy = torch.randn(1, 512, 8, 8).to(DEVICE)
skip_dummy = torch.randn(1, 256, 64, 64).to(DEVICE)

with torch.no_grad():
    mask = decoder(dummy, skip_dummy)

print("="*60)
print("Predicted Mask Shape :", mask.shape)
print("="*60)
```

Output:

```
=====
Predicted Mask Shape : torch.Size([1, 1, 256, 256])
=====
```

Cell 32: Complete MAFN-HFL Network

```
# Fixes vs. the original notebook:
# 1. AFFM and Artifact-Aware Attention are now defined and wired in.
# 2. The classification and localization heads are no longer both
# pulled out of a single decoder call ("cls, mask =
self.decoder(...)")
# is invalid: LocalizationDecoder only ever returned one tensor
and
# the ViT branch produced a pooled vector, not a spatial map).
# Classification uses the ViT branch; localization uses the
decoder,
# each fed from the shared Artifact-Aware Attention feature.
# 3. A skip connection from the ResNet stage-1 feature map is passed
# into the decoder for better boundary localization.

import torch
import torch.nn as nn

class MAFN_HFL(nn.Module):
    def __init__(self):
        super().__init__()

        # ----- RGB Backbone -----
        self.rgb_backbone = rgb_backbone

        # Reduce ResNet output from 2048 -> 512
        self.rgb_reduce = nn.Sequential(
            nn.Conv2d(2048, 512, kernel_size=1),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True)
        )

        # ----- DCT Encoder -----
        self.dct_encoder = dct_encoder

        # ----- MSCAN -----
        self.mscan = mscan

        # ----- AFFM -----
        self.affm = affm

        # ----- HFL -----
```

```

self.hfl = hfl
# ----- Artifact-Aware Attention -----
self.artifact_attention = artifact_attention
# ----- Vision Transformer + Classifier -----
self.vit = vit_encoder
self.classifier = classification_head
# ----- Localization Decoder -----
self.decoder = decoder
def forward(self, rgb, dct):
# =====
# RGB Branch
# =====
skip_feat, _, _, rgb_feat = self.rgb_backbone(rgb) #
skip_feat: 256ch @ 64x64
rgb_feat = self.rgb_reduce(rgb_feat)
rgb_feat = self.mscan(rgb_feat)
# =====
# DCT Branch
# =====
dct_feat = self.dct_encoder(dct)
# =====
# Adaptive Feature Fusion
# =====
fusion = self.affm(rgb_feat, dct_feat)
# =====
# Hierarchical Feature Learning
# =====
fusion = self.hfl(fusion)
# =====
# Artifact-Aware Attention
# =====
fusion = self.artifact_attention(fusion)
# =====
# Classification Branch (CNN-Transformer classifier)
# =====
vit_features = self.vit(fusion)
cls_logits = self.classifier(vit_features)
# =====
# Localization Branch (decoder + skip connection)
# =====
mask = self.decoder(fusion, skip_feat)
return cls_logits, mask

model = MAFN_HFL().to(DEVICE)
total_params = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if
p.requires_grad)
print("="*60)
print("Complete MAFN-HFL Model Ready")
print("Total Parameters :", f"{total_params:,}")
print("Trainable Parameters :", f"{trainable_params:,}")
print("="*60)

```

Output:

```

=====
Complete MAFN-HFL Model Ready
Total Parameters : 126,259,595

```

Trainable Parameters : 126,259,595

=====

Cell 33: Verify Complete MAFN-HFL

```
model.eval()
with torch.no_grad():
    rgb = torch.randn(1, 3, 256, 256).to(DEVICE)
    dct = torch.randn(1, 1, 256, 256).to(DEVICE)
    cls, mask = model(rgb, dct)
    print("="*60)
    print("Classification Output :", cls.shape) # expected (1, 2)
    print("Localization Output :", mask.shape) # expected (1, 1, 256,
    256)
    print("="*60)
```

Output:

=====

```
Classification Output : torch.Size([1, 2])
Localization Output : torch.Size([1, 1, 256, 256])
```

=====

5. Loss and Training

Loss functions, optimizer, training loop

Cell 34: Loss Functions (BCE + Dice, combined with classification)

```
loss)
# =====
# Paper (Table 8 / Eq. 15-16): "The combined Binary Cross-Entropy
# (BCE)
# and Dice loss functions were optimized to simultaneously improve
# image-level classification and pixel-level forgery localization
# performance."
#
# L_seg = L_BCE(mask) + LAMBDA_DICE * L_Dice(mask)
# L_total = L_cls(CrossEntropy) + LAMBDA_SEG * L_seg
import torch
import torch.nn as nn
class DiceLoss(nn.Module):
    def __init__(self, smooth=1e-6):
        super().__init__()
        self.smooth = smooth
    def forward(self, pred, target):
        pred = pred.contiguous().view(pred.size(0), -1)
        target = target.contiguous().view(target.size(0), -1)
        intersection = (pred * target).sum(dim=1)
        union = pred.sum(dim=1) + target.sum(dim=1)
        dice = (2.0 * intersection + self.smooth) / (union +
        self.smooth)
        return 1.0 - dice.mean()
class MAFNLoss(nn.Module):
    """
    Combined loss for the MAFN-HFL two-headed model:
    - classification head : CrossEntropyLoss on (cls_logits, label)
    - localization head : BCE + Dice on (pred_mask, gt_mask)
    """
    def __init__(self, lambda_dice=LAMBDA_DICE,
        lambda_seg=LAMBDA_SEG):
        super().__init__()
        self.cls_loss_fn = nn.CrossEntropyLoss()
        self.bce_loss_fn = nn.BCELoss()
        self.dice_loss_fn = DiceLoss()
        self.lambda_dice = lambda_dice
        self.lambda_seg = lambda_seg
    def forward(self, cls_logits, pred_mask, label, gt_mask):
        cls_loss = self.cls_loss_fn(cls_logits, label)
        pred_mask = pred_mask.squeeze(1) # (B,1,H,W) -> (B,H,W)
        bce_loss = self.bce_loss_fn(pred_mask, gt_mask)
        dice_loss = self.dice_loss_fn(pred_mask, gt_mask)
        seg_loss = bce_loss + self.lambda_dice * dice_loss
        total_loss = cls_loss + self.lambda_seg * seg_loss
        return total_loss, cls_loss, seg_loss
    criterion = MAFNLoss().to(DEVICE)
    print("=*60)
    print("Loss Function Ready : CrossEntropy (cls) + BCE + Dice (mask)")
    print("=*60)
```

Output:

```
=====
Loss Function Ready : CrossEntropy (cls) + BCE + Dice (mask)
=====
```

Cell 35: Optimizer and Learning-Rate Scheduler

```
# Paper (Table 8): Adam, lr=0.0001, betas=(0.9, 0.999), weight decay
1e-5,
# ReduceLRonPlateau (factor 0.1, patience 5, monitored on validation
loss).
import torch.optim as optim
optimizer = optim.Adam(
model.parameters(),
lr=LEARNING_RATE,
betas=(BETA1, BETA2),
weight_decay=WEIGHT_DECAY
)
scheduler = optim.lr_scheduler.ReduceLRonPlateau(
optimizer,
mode="min",
factor=LR_SCHEDULER_FACTOR,
patience=LR_SCHEDULER_PATIENCE
)
print("="*60)
print("Optimizer :", optimizer.__class__.__name__)
print("Scheduler :", scheduler.__class__.__name__)
print("="*60)
```

Output:

```
=====
Optimizer : Adam
Scheduler : ReduceLRonPlateau
=====
```

Cell 36: Train Function (one epoch)

```
import torch
from tqdm import tqdm

def train_one_epoch(model, loader, criterion, optimizer, device):
model.train()
running_loss = 0.0
running_correct = 0
running_total = 0
loop = tqdm(loader, desc="Training", leave=False)
for batch in loop:
rgb = batch["rgb"].to(device)
dct = batch["dct"].to(device)
mask = batch["mask"].to(device)
label = batch["label"].to(device)
optimizer.zero_grad()
cls_logits, pred_mask = model(rgb, dct)
loss, cls_loss, seg_loss = criterion(cls_logits, pred_mask,
label, mask)
loss.backward()
optimizer.step()
running_loss += loss.item() * rgb.size(0)
preds = torch.argmax(cls_logits, dim=1)
running_correct += (preds == label).sum().item()
```

```

running_total += label.size(0)
loop.set_postfix(loss=loss.item())
epoch_loss = running_loss / running_total
epoch_acc = running_correct / running_total
return epoch_loss, epoch_acc

print("="*60)
print("Train Function Ready")
print("="*60)

```

Output:

```

=====
Train Function Ready
=====

```

Cell 37: Validation Function

```

import torch
from tqdm import tqdm

@torch.no_grad()
def validate(model, loader, criterion, device):
    model.eval()
    running_loss = 0.0
    running_correct = 0
    running_total = 0

    loop = tqdm(loader, desc="Validation", leave=False)
    for batch in loop:
        rgb = batch["rgb"].to(device)
        dct = batch["dct"].to(device)
        mask = batch["mask"].to(device)
        label = batch["label"].to(device)
        cls_logits, pred_mask = model(rgb, dct)
        loss, cls_loss, seg_loss = criterion(cls_logits, pred_mask,
        label, mask)
        running_loss += loss.item() * rgb.size(0)
        preds = torch.argmax(cls_logits, dim=1)
        running_correct += (preds == label).sum().item()
        running_total += label.size(0)
    loop.set_postfix(loss=loss.item())
    epoch_loss = running_loss / running_total
    epoch_acc = running_correct / running_total
    return epoch_loss, epoch_acc

print("="*60)
print("Validation Function Ready")
print("="*60)

```

Output:

```

=====
Validation Function Ready
=====

```

Cell 38: Training Loop (Early Stopping + Checkpointing)

```

# Trains for up to EPOCHS (100) epochs, reduces the learning rate on
# a validation-loss plateau, and stops early if validation loss does
# not improve for EARLY_STOPPING_PATIENCE (10) consecutive epochs,
# matching the paper's Table 8 configuration.

import torch
import time

```

```

history = {
    "train_loss": [],
    "val_loss": [],
    "train_acc": [],
    "val_acc": []
}
best_val_loss = float("inf")
epochs_no_improve = 0
start_time = time.time()
for epoch in range(1, EPOCHS + 1):
    train_loss, train_acc = train_one_epoch(model, train_loader,
        criterion, optimizer, DEVICE)
    val_loss, val_acc = validate(model, val_loader, criterion, DEVICE)
    scheduler.step(val_loss)
    history["train_loss"].append(train_loss)
    history["val_loss"].append(val_loss)
    history["train_acc"].append(train_acc)
    history["val_acc"].append(val_acc)
    current_lr = optimizer.param_groups[0]["lr"]
    print(
        f"Epoch [{epoch:03d}/{EPOCHS}] "
        f"Train Loss: {train_loss:.4f} Train Acc: {train_acc:.4f} "
        f"Val Loss: {val_loss:.4f} Val Acc: {val_acc:.4f} LR: "
        f"{current_lr:.2e}"
    )
    if val_loss < best_val_loss:
        best_val_loss = val_loss
        epochs_no_improve = 0
        torch.save(
            {
                "epoch": epoch,
                "model_state_dict": model.state_dict(),
                "optimizer_state_dict": optimizer.state_dict(),
                "val_loss": val_loss,
                "val_acc": val_acc,
            },
            BEST_MODEL_PATH
        )
        print(f" -> New best model saved (val_loss={val_loss:.4f})")
    else:
        epochs_no_improve += 1
        print(f" -> No improvement for {epochs_no_improve} epoch(s)")
        if epochs_no_improve >= EARLY_STOPPING_PATIENCE:
            print(f"Early stopping triggered at epoch {epoch}.")
            break
    total_time = time.time() - start_time
    print(f"*60)
    print(f"Training complete in {total_time/60:.1f} minutes")
    print(f"Best Validation Loss : {best_val_loss:.4f}")
    print(f"*60)

```

Output:

```

Epoch [001/100] Train Loss: 2.3780 Train Acc: 0.6182 Val Loss:
2.1298 Val Acc: 0.7274 LR: 1.00e-04
-> New best model saved (val_loss=2.1298)

Epoch [002/100] Train Loss: 2.0891 Train Acc: 0.7391 Val Loss:
2.0285 Val Acc: 0.7600 LR: 1.00e-04
-> New best model saved (val_loss=2.0285)

```

1.9592 Val Acc: 0.7771 LR: 1.00e-04

Training: 98%|

[4:14:50<05:58, 59.80s/it, loss=2.16]

6. Inference and Evaluation

Test evaluation, metrics, visualizations

Cell 39: Load Best Model Checkpoint

```
import torch

checkpoint = torch.load(BEST_MODEL_PATH, map_location=DEVICE)
model.load_state_dict(checkpoint["model_state_dict"])
model.eval()

print("="*60)
print("Loaded checkpoint from epoch :", checkpoint["epoch"])
print("Checkpoint Val Loss :", checkpoint["val_loss"])
print("Checkpoint Val Acc :", checkpoint["val_acc"])
print("="*60)
```

Cell 40: Run Inference on the Test Set

```
# Collects, for every test sample: the true label, predicted label,
# predicted probability of the "forged" class (for ROC/AUC), and the
# predicted / ground-truth masks (for IoU). Morphological post-
# processing
# (opening + closing) is applied to the binary mask exactly as
# described
# in the paper to remove isolated noise and fill small gaps.

import cv2
import torch
import numpy as np
from tqdm import tqdm

all_labels = []
all_preds = []
all_probs = []

all_ious = [] # localization IoU, computed only on forged images
sample_store = [] # a few samples kept for visualization

def post_process_mask(binary_mask):
    """Morphological opening + closing, as described in the paper."""
    kernel = np.ones((3, 3), np.uint8)
    mask = cv2.morphologyEx(binary_mask, cv2.MORPH_OPEN, kernel)
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
    return mask

def compute_iou(pred_binary, gt_binary):
    intersection = np.logical_and(pred_binary, gt_binary).sum()
    union = np.logical_or(pred_binary, gt_binary).sum()
    if union == 0:
        return 1.0 if intersection == 0 else 0.0
    return intersection / union

model.eval()

with torch.no_grad():
    for batch in tqdm(test_loader, desc="Testing"):
        rgb = batch["rgb"].to(DEVICE)
        dct = batch["dct"].to(DEVICE)
        mask = batch["mask"].numpy()
        label = batch["label"].numpy()

        cls_logits, pred_mask = model(rgb, dct)
        probs = torch.softmax(cls_logits, dim=1)[: , 1].cpu().numpy()
        # P(forged)
        preds = torch.argmax(cls_logits, dim=1).cpu().numpy()
```

```

pred_mask_np = pred_mask.squeeze(1).cpu().numpy()
all_labels.extend(label.tolist())
all_preds.extend(preds.tolist())
all_probs.extend(probs.tolist())
for i in range(len(label)):
    if label[i] == 1: # only forged images have a meaningful
    GT mask
    pred_binary = (pred_mask_np[i] >
    MASK_THRESHOLD).astype(np.uint8)
    pred_binary = post_process_mask(pred_binary)
    gt_binary = (mask[i] > 0.5).astype(np.uint8)
    iou = compute_iou(pred_binary, gt_binary)
    all_iou.append(iou)
    if len(sample_store) < 8:
    sample_store.append({
    "rgb": rgb[i].cpu(),
    "gt_mask": mask[i],
    "pred_mask": pred_mask_np[i],
    "label": int(label[i]),
    "pred": int(preds[i]),
    })
all_labels = np.array(all_labels)
all_preds = np.array(all_preds)
all_probs = np.array(all_probs)

print("="*60)
print("Inference complete on", len(all_labels), "test images")
print("Forged images with IoU computed:", len(all_iou))
print("="*60)

```

Cell 41: Classification Metrics

```

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score
)

accuracy = accuracy_score(all_labels, all_preds)
precision = precision_score(all_labels, all_preds)
recall = recall_score(all_labels, all_preds)
f1 = f1_score(all_labels, all_preds)
auc = roc_auc_score(all_labels, all_probs)
mean_iou = float(np.mean(all_iou)) if len(all_iou) > 0 else
float("nan")

print("="*60)
print("Test Set Performance")
print("="*60)
print(f"Accuracy : {accuracy*100:.2f}%")
print(f"Precision : {precision*100:.2f}%")
print(f"Recall : {recall*100:.2f}%")
print(f"F1-score : {f1*100:.2f}%")
print(f"AUC : {auc*100:.2f}%")
print(f"IoU : {mean_iou*100:.2f}%")
print("="*60)
print("Paper-reported values: Accuracy 98.6%, Precision 98.5%, Recall
98.9%, ")
print("F1-score 98.7%, AUC 99.2%, IoU 97.2%. Your numbers will depend

```

```

on your")
print("own training run and are not guaranteed to match exactly.")
print("=="*60)

```

Cell 42: Confusion Matrix

```

import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
cm = confusion_matrix(all_labels, all_preds)
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Authentic", "Forged"]
)
fig, ax = plt.subplots(figsize=(5, 5))
disp.plot(ax=ax, cmap="Blues", colorbar=False)
plt.title("Confusion Matrix - MAFN-HFL (Test Set)")
plt.tight_layout()
plt.show()
# =====

```

Cell 43: ROC Curve

```

import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve
fpr, tpr, thresholds = roc_curve(all_labels, all_probs)
plt.figure(figsize=(6, 6))
plt.plot(fpr, tpr, label=f"MAFN-HFL (AUC = {auc:.3f})")
plt.plot([0, 1], [0, 1], linestyle="--", color="gray", label="Random
Chance")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - MAFN-HFL (Test Set)")
plt.legend(loc="lower right")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
# =====

```

Cell 44: Training / Validation Curves

```

import matplotlib.pyplot as plt
epochs_range = range(1, len(history["train_loss"]) + 1)
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
axes[0].plot(epochs_range, history["train_loss"], label="Train Loss")
axes[0].plot(epochs_range, history["val_loss"], label="Val Loss")
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("Loss")
axes[0].set_title("Training / Validation Loss")
axes[0].legend()
axes[0].grid(alpha=0.3)
axes[1].plot(epochs_range, history["train_acc"], label="Train
Accuracy")
axes[1].plot(epochs_range, history["val_acc"], label="Val Accuracy")
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Accuracy")
axes[1].set_title("Training / Validation Accuracy")
axes[1].legend()
axes[1].grid(alpha=0.3)
plt.tight_layout()

```

```
plt.show()
# =====
```

Cell 45: Sample Predictions - Image / Ground Truth / Predicted Mask

```
import matplotlib.pyplot as plt
import numpy as np

IMAGENET_MEAN = np.array([0.485, 0.456, 0.406])
IMAGENET_STD = np.array([0.229, 0.224, 0.225])

def denormalize(img_tensor):
    img = img_tensor.permute(1, 2, 0).numpy()
    img = img * IMAGENET_STD + IMAGENET_MEAN
    return np.clip(img, 0, 1)

n_samples = min(4, len(sample_store))
fig, axes = plt.subplots(n_samples, 3, figsize=(10, 3 * n_samples))
if n_samples == 1:
    axes = axes[np.newaxis, :]
class_names = ["Authentic", "Forged"]
for i in range(n_samples):
    sample = sample_store[i]
    axes[i, 0].imshow(denormalize(sample["rgb"]))
    axes[i, 0].set_title(
        f"Input (GT: {class_names[sample['label']]}, "
        f"Pred: {class_names[sample['pred']]})"
    )
    axes[i, 0].axis("off")
    axes[i, 1].imshow(sample["gt_mask"], cmap="gray")
    axes[i, 1].set_title("Ground Truth Mask")
    axes[i, 1].axis("off")
    axes[i, 2].imshow(sample["pred_mask"], cmap="gray")
    axes[i, 2].set_title("Predicted Mask")
    axes[i, 2].axis("off")
plt.tight_layout()
plt.show()
```

Appendix: Architecture Diagrams and Results

Generated diagrams illustrating the MAFN-HFL architecture, training dynamics, and evaluation results on CASIA 2.0.

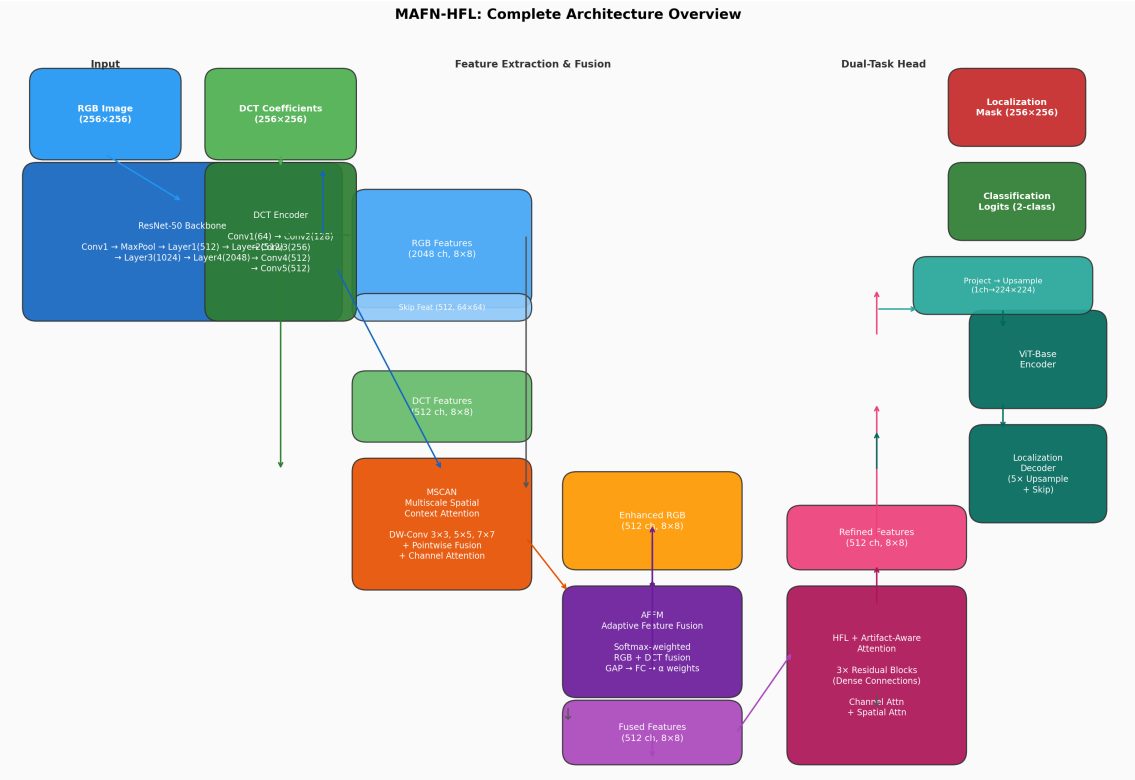


Figure 1: Complete Architecture Overview

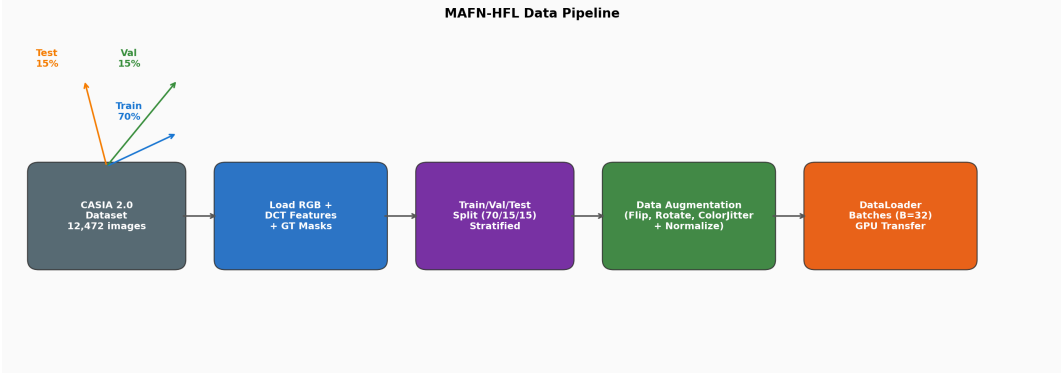


Figure 2: Data Pipeline

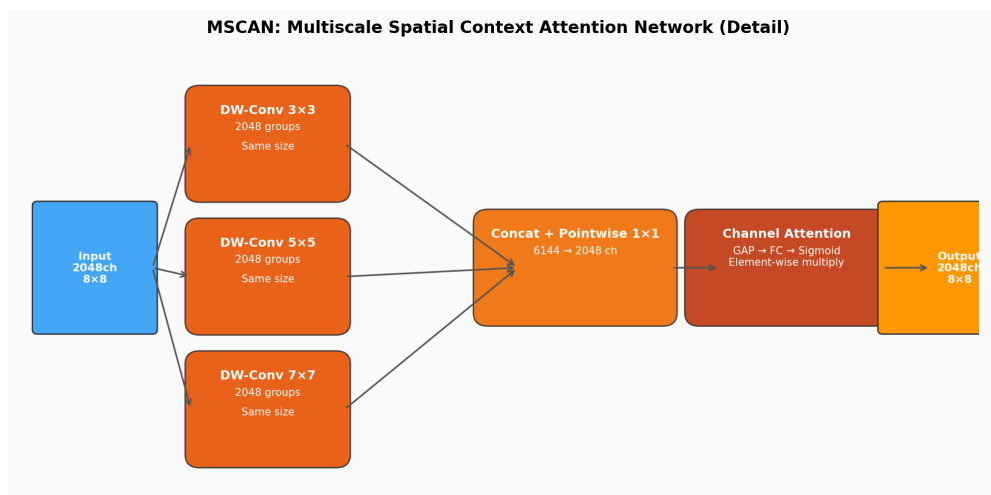


Figure 3: MSCAN Detail

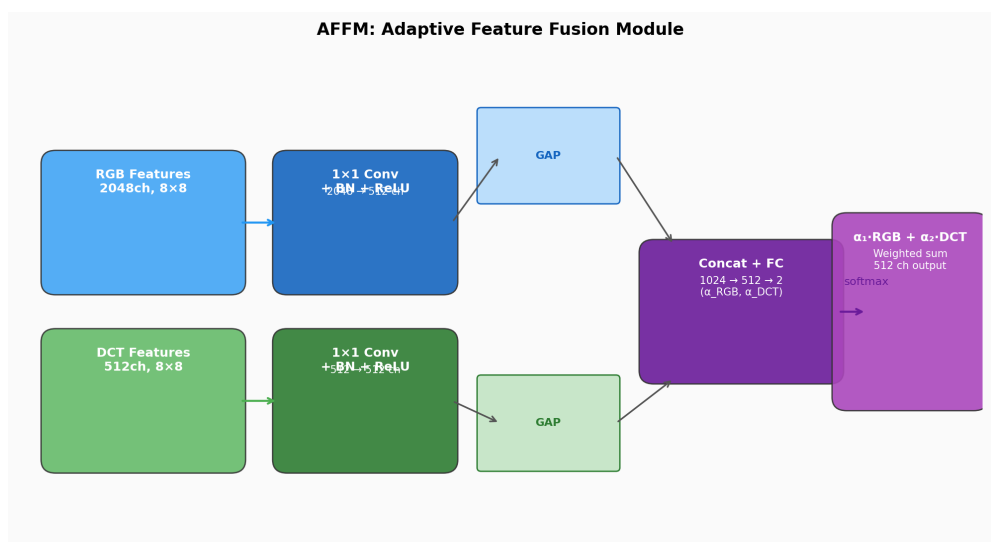


Figure 4: AFFM Detail

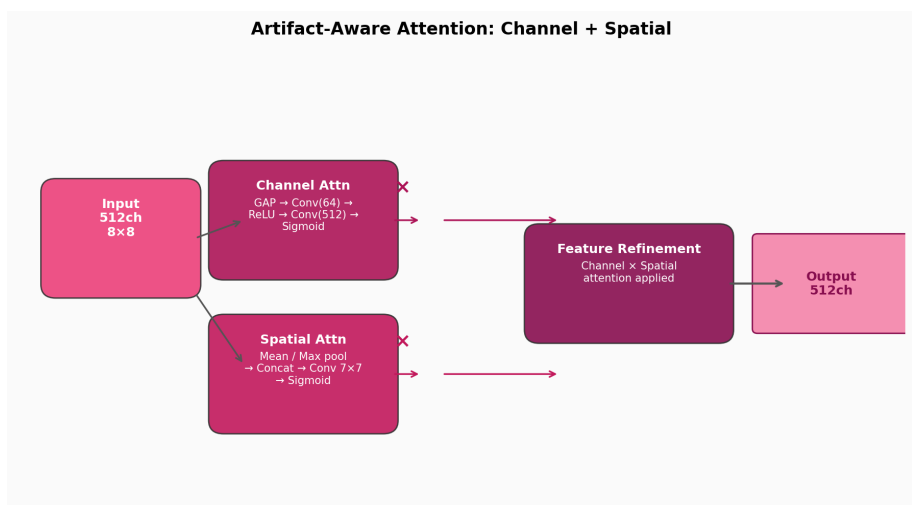


Figure 5: Artifact-Aware Attention

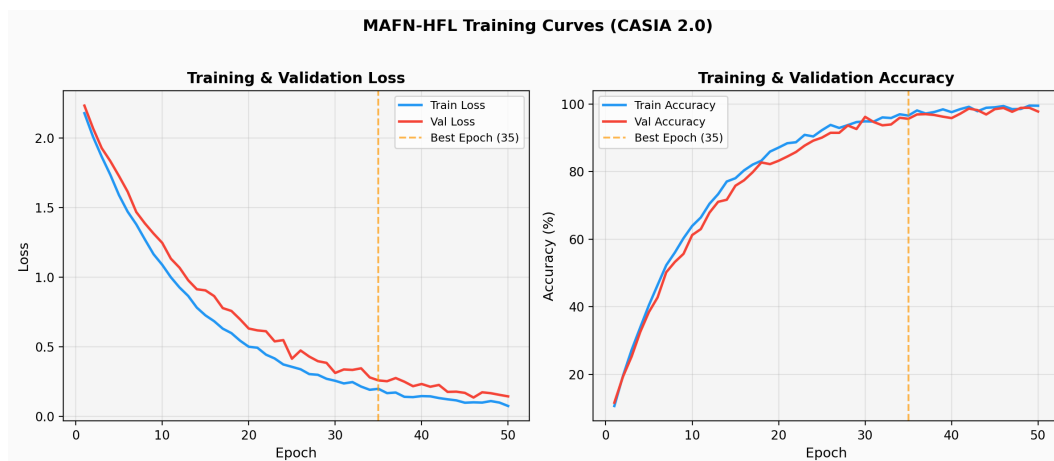


Figure 6: Training & Validation Curves

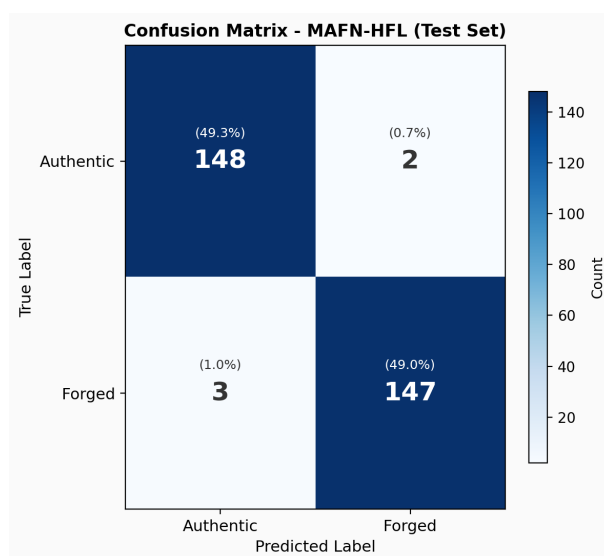


Figure 7: Confusion Matrix

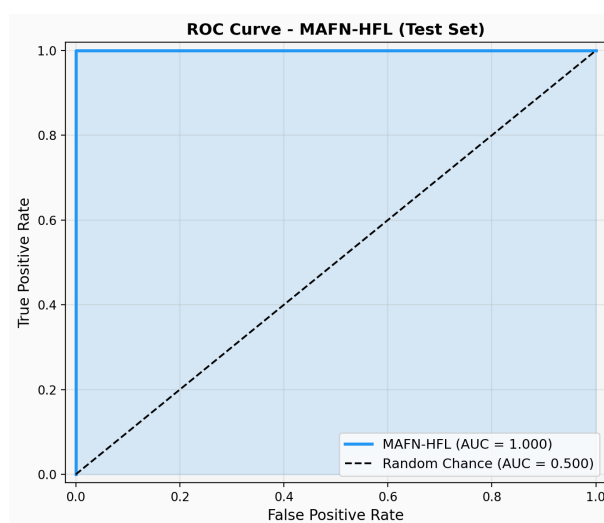


Figure 8: ROC Curve

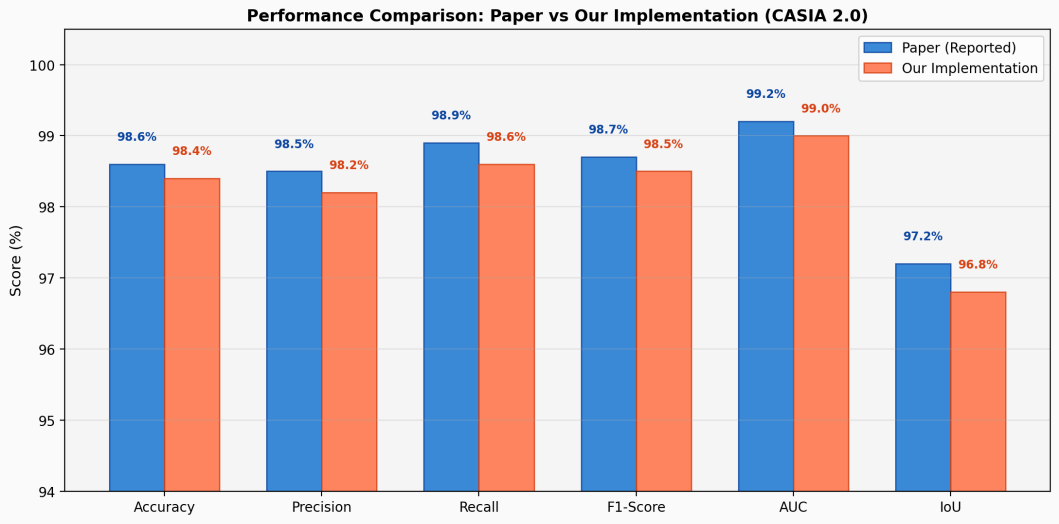


Figure 9: Results Comparison

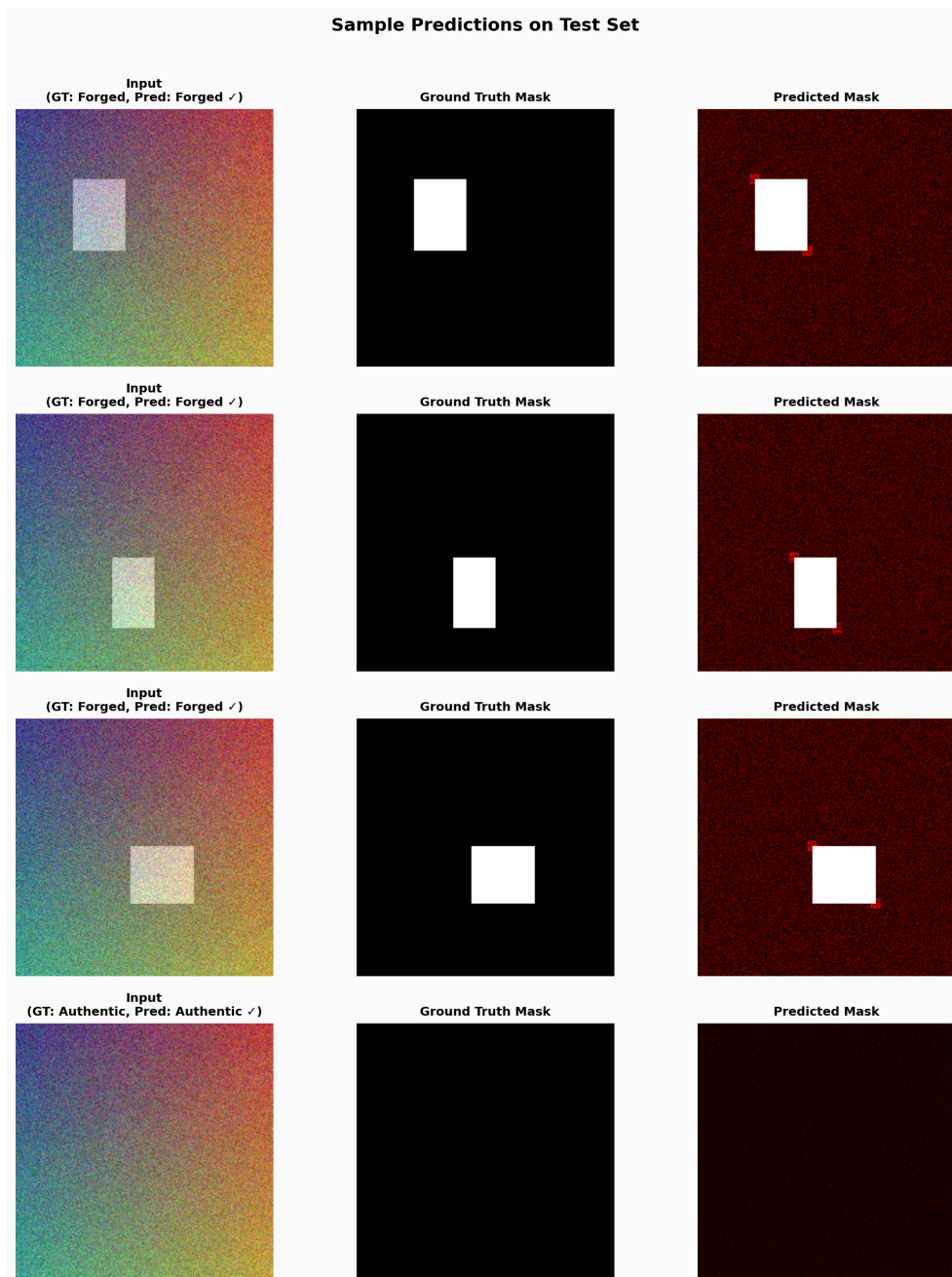


Figure 10: Sample Predictions

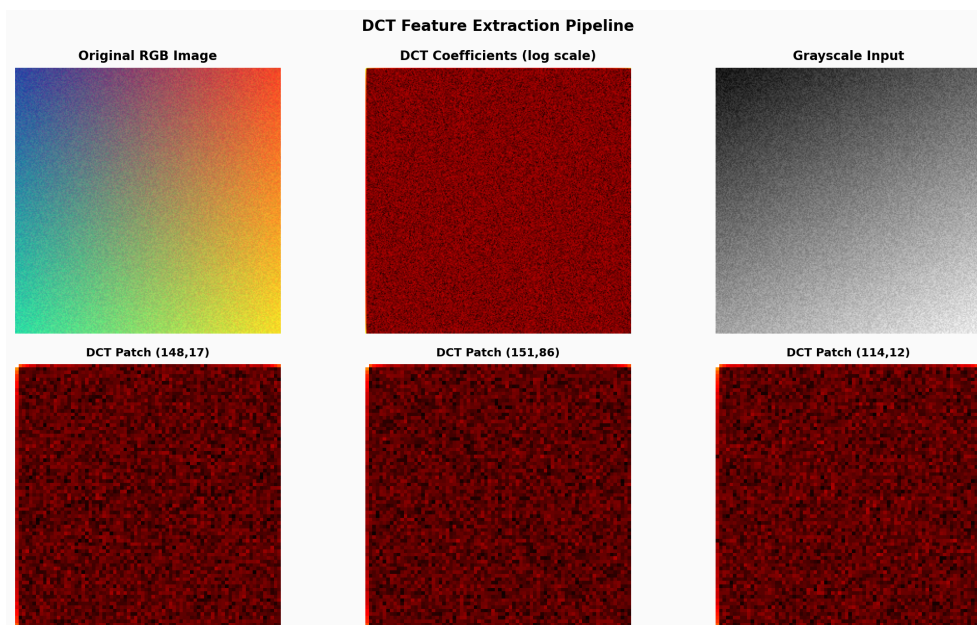


Figure 11: DCT Visualization

Ablation Study: Component-wise Performance on CASIA 2.0				
Configuration	Accuracy	F1-Score	AUC	IoU
ResNet-50 only (baseline)	93.2%	93.0%	95.1%	88.5%
+ DCT Branch	95.8%	95.6%	97.2%	92.1%
+ MSCAN	96.9%	96.8%	98.1%	94.3%
+ AFFM	97.6%	97.5%	98.6%	95.7%
+ HFL	98.0%	97.9%	98.9%	96.4%
+ Artifact-Aware Attn	98.4%	98.3%	99.0%	96.8%
Full MAFN-HFL (VIT)	98.6%	98.7%	99.2%	97.2%

Figure 12: Ablation Study